

Day 28: Zephyr Synchronization Primitives

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

k_mutex (Recursive-safe)

A **mutex** provides mutual exclusion — only one thread holds it at a time. Zephyr's mutex supports **priority inheritance** (PI) to prevent priority inversion:

```
K_MUTEX_DEFINE(spi_mutex);    // Static definition

k_mutex_lock(&spi_mutex, K_FOREVER); // Block until locked
// ... critical section: SPI transfer ...
k_mutex_unlock(&spi_mutex);
```

Unlike a semaphore, a mutex can only be unlocked by the thread that locked it. Zephyr's mutex is **recursive** — the same thread can lock it multiple times (must unlock the same number of times).

k_sem (Counting & Binary Semaphore)

A **semaphore** is used for signaling — one thread (or ISR) signals, another thread waits:

```
K_SEM_DEFINE(data_ready, 0, 1); // Initial=0, Max=1 (binary semaphore)
K_SEM_DEFINE(pool_available, 4, 4); // Initial=4, Max=4 (counting semaphore)

// Signaler (ISR or producer thread):
k_sem_give(&data_ready);

// Waiter (consumer thread):
k_sem_take(&data_ready, K_FOREVER); // Block until signaled
```

Binary semaphore (max=1): simple flag signaling. Counting semaphore (max=N): controls access to N identical resources.

k_condvar (Condition Variables)

A **condition variable** lets a thread wait until a specific condition becomes true, atomically releasing a mutex:

```
K_MUTEX_DEFINE(cond_mutex);
K_CONDVAR_DEFINE(data_available);

// Waiter:
k_mutex_lock(&cond_mutex, K_FOREVER);
while (!data_ready) {
    k_condvar_wait(&data_available, &cond_mutex, K_FOREVER);
}
process_data();
k_mutex_unlock(&cond_mutex);

// Signaler:
k_mutex_lock(&cond_mutex, K_FOREVER);
data_ready = true;
k_condvar_signal(&data_available);
k_mutex_unlock(&cond_mutex);
```

k_poll for Multi-Event Waiting

k_poll blocks until **any one of multiple events** is ready — like **select()** for RTOS primitives:

```
struct k_poll_event events[] = {
    K_POLL_EVENT_STATIC_INITIALIZER(K_POLL_TYPE_SEM_AVAILABLE, K_POLL_MODE_AUTO_RESET, &sem_a, 0),
    K_POLL_EVENT_STATIC_INITIALIZER(K_POLL_TYPE_SEM_AVAILABLE, K_POLL_MODE_AUTO_RESET, &sem_b, 0),
};
k_poll(events, 2, K_FOREVER); // Block until sem_a OR sem_b is signaled
if (events[0].state == K_POLL_STATE_SEM_AVAILABLE) { /* sem_a triggered */ }
```

Priority Inversion & PI Mutex

Priority inversion: low-priority thread L holds a mutex needed by high-priority thread H. A medium-priority thread M preempts L but not H. H is blocked waiting for L, but L can't run because M is running. Result: H (highest priority) is blocked by M (medium priority).

Priority inheritance (PI) mutex solves this: when H waits for the mutex held by L, Zephyr temporarily raises L's priority to H's priority, allowing L to complete and release the mutex quickly.

Zephyr enables PI mutex automatically when `CONFIG_PRIORITY_CEILING=y`.

RAII Wrappers for Zephyr Sync

Wrap Zephyr sync primitives in RAII classes (as shown in Day 16) for safe, exception-safe locking:

```
class ScopedLock {
    k_mutex &m_;
public:
    ScopedLock(k_mutex &m) : m_(m) { k_mutex_lock(&m_, K_FOREVER); }
    ~ScopedLock()                { k_mutex_unlock(&m_); }
    ScopedLock(const ScopedLock &) = delete;
};
```

Practice Tasks

1. Create a producer-consumer pair: producer thread fills a `k_msgq`, consumer drains it, measure throughput
2. Demonstrate priority inversion: create H, M, L threads and a shared mutex — add a PI mutex and compare
3. Use `k_poll` to wait on a button semaphore AND a timeout — handle both cases differently
4. Implement a thread-safe log buffer: any thread can write, a dedicated log thread drains and prints

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day28
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day28/main.cpp](#)